

Actividad Evaluativa 2

Integración JS y metodologías web en Gym Control

CARRERA: Ingeniería en Sistemas de Información

MATERIA: Paradigmas y Lenguajes de Programación III

COMISIÓN: A

PROFESOR: Sicardi, Dante

ESTUDIANTES: Martins, Santino; Olexyn, Franco; Tarnoski, Santiago

FECHA: 29-09-2026

REPOSITORIO: <https://github.com/santiagotarnoski/gym-control-ae1>



INTRODUCCIÓN

Gym Control es un sitio de consulta de planes de gimnasio desarrollado en la Actividad Evaluativa 1. En esta Actividad Evaluativa 2 cada integrante incorpora un módulo con JavaScript según la tarjeta que le fue asignada y lo fundamenta con una metodología de la Unidad 1. Los módulos conservan los cuatro planes, las páginas y el diseño del proyecto previo.

Todas las tarjetas comparten un requisito transversal: capturar los eventos con `addEventListener` en `js/app.js`, sin atributos `onclick` ni `onsubmit` en el HTML, y cargar o enviar datos con `fetch` y `async/await` sobre un archivo JSON local. Cada módulo ocupa un bloque independiente de `js/app.js`, de modo que las partes conviven sin interferir.

Asignación de tarjetas

Integrante	Tarjeta asignada	Metodología de la Unidad 1
Tarnoski, Santiago	3. Buscador y filtro de catálogo en tiempo real	SOHDM
Olexyn, Franco	4. Módulo de registro y autenticación de usuarios	VSDM
Martins, Santino	11. Formulario de reseñas, opiniones o reclamos	RNA

El objetivo de este informe es documentar el comportamiento de cada módulo, su relación con el proyecto previo, su fundamentación metodológica y las evidencias de verificación. El alcance es académico: los datos son de ejemplo y no hay persistencia, contratación ni autenticación real.



DESARROLLO

1 TARJETA 3: BUSCADOR Y FILTRO DE CATÁLOGO EN TIEMPO REAL

Responsable: Tarnoski, Santiago

La tarjeta 3 corresponde a la opción Buscador y filtro de catálogo en tiempo real. Exige registrar los eventos con `addEventListener`, cargar el catálogo completo mediante `fetch` y `async/await`, filtrar los datos en memoria con `Array.filter` y presentar los resultados en el DOM. La fundamentación de la Unidad 1 corresponde a SOHDM, aplicada al escenario de exploración y búsqueda.

El módulo amplía el catálogo en tarjetas con un buscador y un filtro por duración. Conserva los cuatro planes existentes y permite localizar una opción antes de acceder al formulario de selección.

1.1 Alcance y trazabilidad

Requisito	Evidencia en la implementación
Continuidad con AE1	Los cuatro planes, precios, páginas y estilos de Gym Control permanecen.
Eventos desacoplados	<code>input</code> , <code>change</code> y <code>click</code> se registran en <code>js/app.js</code> con <code>addEventListener</code> .
HTTP y asincronía	<code>fetch</code> y <code>async/await</code> cargan <code>data/planes.json</code> desde un servidor local.
Búsqueda dinámica	<code>Array.filter</code> combina texto y duración; el DOM presenta tarjetas y contador.
Metodología SOHDM	Escenario E01, objetos y mapa navegacional de consulta y selección.

1.2 Descripción del módulo

El usuario accede a `listado_box.html` desde la portada o desde el listado en tabla. Antes de ofrecer la búsqueda, `js/app.js` consulta por HTTP el archivo `data/planes.json`. Cada registro describe el identificador, nombre, duración, precio, categoría, descripción y marca de



destacado de un plan. Los precios se conservan del proyecto original: Mensual \$25.000, Trimestral \$67.500, Semestral \$120.000 y Anual \$216.000.

El campo de texto busca coincidencias en nombre, descripción y categoría. La selección de duración admite 1, 3, 6 y 12 meses. Ambos criterios se aplican en conjunto. El usuario puede cambiar el texto sin pulsar un botón, limpiar los filtros o elegir un resultado. Por ejemplo, escribir renovacion encuentra renovación sin exigir la tilde. El término mensual devuelve también el Plan Anual porque su descripción menciona valor mensual.

1.3 Organización técnica

Componente	Responsabilidad
listado_box.html	Presenta controles, estado de carga, contenedor y mensaje sin resultados.
data/planes.json	Almacena los cuatro planes de ejemplo de la AE1.
js/app.js	Carga, valida, filtra, crea tarjetas y conserva el plan al navegar.
css/styles.css	Mantiene el diseño y adapta los controles a la vista móvil.
comprar.html	Recibe el identificador del plan y conserva las funciones previas.

La carga usa `fetch("data/planes.json")` con `await`. El programa verifica `response.ok` y valida el formato antes de habilitar los filtros. Un error HTTP, de red o de datos muestra un mensaje y un botón Reintentar. El catálogo vacío tiene un mensaje diferente del caso sin coincidencias. Al modificar los criterios, el programa aplica `Array.filter` sobre los registros ya cargados, sin realizar otra petición [2, 3].

Las tarjetas se construyen con `createElement` y `textContent`, y se insertan con `replaceChildren`. Esta decisión permite presentar el contenido JSON como texto. Los archivos originales `js/subtotal.js` y `js/cupon.js` no se modificaron; al seguir un resultado, un parámetro `plan` preselecciona el radio correspondiente y dispara su evento `change` para actualizar el subtotal existente.



1.4 Escenario E01 de exploración y búsqueda

Actor principal: visitante o socio interesado en comparar planes. Precondición: el sitio se sirve mediante HTTP y el catálogo puede cargarse. Objetivo: encontrar un plan compatible con los criterios del usuario y continuar a su selección.

Paso	Acción y respuesta esperada
1	Abrir Planes en cards; el sistema carga y presenta cuatro tarjetas.
2	Escribir progreso; aparece el Plan Semestral por su categoría.
3	Elegir 6 meses; permanece el resultado compatible.
4	Elegir el plan; se abre <code>comprar.html?plan=semestral</code> .
5	Consultar el formulario; el plan aparece marcado y el subtotal se calcula.

Flujos alternativos: si los filtros no encuentran registros, se muestra cero resultados y se permite modificar o limpiar la consulta. Si falla la carga, se bloquean los controles y se ofrece reintento. Si el JSON está vacío, se indica que no hay planes disponibles. La postcondición del flujo principal es una selección visual de plan; no se asigna una membresía.

1.5 Fundamentación de SOHDM

SOHDM (Scenario Based Object Oriented Hypermedia Design Methodology) usa escenarios para identificar requisitos de sistemas hipermedia y emplea vistas de objetos para definir unidades navegacionales. Lee, Lee y Yoo describen seis fases: análisis del dominio, modelado de objetos, diseño de vistas, diseño de navegación, diseño de implementación y construcción [1].

Fase	Aplicación en Gym Control
Análisis del dominio	Actor, objetivo, flujo principal y alternativas del escenario E01.
Modelado de objetos	Plan, Consulta y Catálogo como objetos conceptuales.
Diseño de vistas	Barra de filtros, tarjetas, contador y estados de error o vacío.



Diseño de navegación	Accesos desde Inicio y tabla; salida a detalle o selección.
Diseño de implementación	JSON, funciones JavaScript, estructura HTML y estilos.
Construcción	Integración del código y pruebas de navegador con servidor HTTP.

Plan aporta los atributos que se buscan y presentan. Consulta representa el texto normalizado y la duración. Catálogo contiene los planes cargados y obtiene las coincidencias. Las vistas muestran únicamente la información necesaria para explorar y decidir. La aplicación usa registros JSON y funciones JavaScript; no implementa un motor formal de SOHDM.

1.6 Mapa navegacional

El mapa deriva del escenario E01. La navegación existente Inicio → Listado en tabla o Catálogo en tarjetas se mantiene. Búsqueda y resultados son estados de la página listado_box.html, no páginas independientes. Desde una tarjeta, el usuario puede ir a la selección con el ID del plan. La ficha producto.html del proyecto previo corresponde específicamente al Plan Mensual.

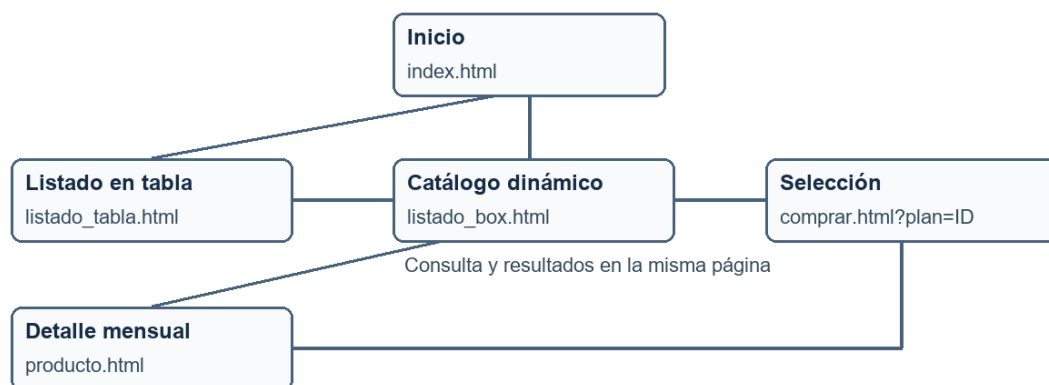


Figura 1 Mapa navegacional del escenario de exploración y búsqueda



Los enlaces de retorno permiten revisar la tabla, modificar filtros o limpiar la consulta. Si falla la carga, Reintentar vuelve a ejecutar la petición HTTP sobre el mismo nodo. El archivo editable docs/mapa-navegacional-tarjeta3.mmd conserva el diagrama.

1.7 Interacción y estados de la interfaz

El evento input del campo de búsqueda y change del selector de duración llaman a filtrarPlanes(). El botón Limpiar filtros restaura ambos controles y devuelve el foco al campo de texto. El botón Reintentar invoca cargarCatalogo(). Todos se vinculan mediante addEventListener en js/app.js; el HTML no contiene atributos onclick ni onsubmit [4].

El contador informa cuántos planes se muestran de los cuatro disponibles. role="status" y aria-live="polite" comunican los resultados; aria-busy indica la carga. Mientras la respuesta no es válida, los controles permanecen deshabilitados. El CSS reorganiza los controles y las tarjetas en una columna para teléfonos.

1.8 Verificación

Se ejecutaron 36 comprobaciones en Google Chrome mediante un servidor HTTP local. El registro completo se conserva en docs/pruebas-tarjeta3.json y el procedimiento reproducible en tests/catalogo.cjs. Las pruebas cubren la carga de las cuatro tarjetas, búsquedas con tildes y mayúsculas, combinación de filtros, cero resultados, limpieza, selección de plan, compatibilidad con subtotal y cupón, vista móvil y fallos HTTP, de red o de JSON.

Caso representativo	Resultado observado
Búsqueda por progreso y 6 meses	Solo se presenta el Plan Semestral.
Búsqueda incompatible	Se informa 0 de 4 planes y se ofrece limpiar.
Plan Trimestral elegido	comprar.html preselecciona el plan y muestra \$67.500.
Error HTTP 404	Los filtros se bloquean; el reintento recupera el catálogo.
Vista de 390 píxeles	No existe desbordamiento horizontal.



El resultado global fue de 36 verificaciones correctas. El contenido del JSON se insertó como texto, y el recorrido normal no produjo errores JavaScript. Las capturas de pantalla se incluyen en Anexos.



2 TARJETA 4: REGISTRO Y AUTENTICACIÓN DE SOCIOS

Responsable: Olexyn, Franco

La tarjeta 4 corresponde a la opción Módulo de registro y autenticación de usuarios. Exige capturar el envío de los formularios con `addEventListener`, validar la coincidencia y la longitud mínima de las claves, y simular con `fetch` la verificación de credenciales con una respuesta visible en pantalla. La fundamentación de la Unidad 1 corresponde a VSDM, que define qué ve un visitante y qué ve un usuario autenticado.

En Gym Control el usuario autenticado es el socio. El análisis de la AE1 ya preveía los requisitos RF01 Autenticación, RF02 Roles y permisos, RF07 Estados de membresía y RF14 Portal del socio. Esta entrega implementa un recorte de esos requisitos: el socio ingresa, consulta su membresía y continúa a la renovación, mientras que el visitante conserva el acceso público al catálogo. Como en la tarjeta 3, los datos son de ejemplo y no hay persistencia real.

2.1 Alcance y trazabilidad

Requisito	Evidencia en la implementación
Continuidad con AE1 y tarjeta 3	Se agrega <code>acceso.html</code> ; el catálogo, el subtotal y el cupón no cambian.
Eventos desacoplados	<code>submit</code> , <code>input</code> y <code>click</code> se registran en <code>js/app.js</code> con <code>addEventListener</code> ; el HTML no usa <code>onclick</code> ni <code>onsubmit</code> .
HTTP y asincronía	<code>fetch</code> y <code>async/await</code> consultan <code>data/usuarios.json</code> y <code>data/planes.json</code> .
Validación de claves	Campos obligatorios con <code>trim</code> , formato de e-mail, mínimo de 8 caracteres y coincidencia de claves.
Respuesta visual	Mensajes de error o de éxito, campos marcados y cambio de vista sin recargar la página.
Metodología VSDM	Clases Visitante y Socio, matriz de visibilidad y diagrama de vistas.



2.2 Descripción del módulo de acceso

La página acceso.html se abre desde el enlace Ingresar, presente en el encabezado de todas las páginas. Para el visitante muestra dos formularios: Ingresar, con e-mail y clave, y Crear cuenta, con nombre, e-mail, clave y repetición de la clave. Los campos de clave usan type="password" y los valores de autocomplete adecuados para que el navegador pueda ofrecer su gestor de contraseñas.

Cuando las credenciales son válidas, la misma página reemplaza los formularios por la vista del socio: saludo, número de socio, plan, estado de la membresía, fecha de vencimiento y días restantes. El estado se calcula a partir del vencimiento: Activa, Por vencer cuando faltan siete días o menos, Vencida o Sin membresía. El botón Renovar plan abre comprar.html con el plan del socio marcado y con el nombre y el e-mail precargados; Cerrar sesión devuelve la vista de visitante.

Para la demostración, data/usuarios.json contiene dos socios de prueba: uno con el Plan Semestral activo y otro con el Plan Mensual vencido. La clave de ambos figura en el README del repositorio. El registro de una cuenta nueva comprueba que el e-mail no exista e inicia la sesión como socio sin membresía, pero no agrega datos al archivo.

2.3 Organización técnica

Componente	Responsabilidad
acceso.html	Formularios de ingreso y registro, vista del socio y zonas de mensajes con role="status".
data/usuarios.json	Socios de prueba con identificador, nombre, e-mail, sal, resumen SHA-256 de la clave y membresía.
js/app.js	Bloque propio de la tarjeta 4: validación, verificación simulada, sesión y cambio de vistas.
css/styles.css	Formularios, estados de la membresía, campos con error y menú en dos filas en teléfonos.
Páginas existentes	Enlace Ingresar en el encabezado; comprar.html recibe los datos del socio.



El bloque de la tarjeta 4 se agregó al final de `js/app.js` como una función autoejecutable separada de la tarjeta 3, de modo que ambas comparten el archivo sin interferir. Las funciones `consultarSocios` y `verificarCredenciales` representan al servidor: obtienen el archivo con `fetch`, verifican `response.ok` y el formato de los datos, y devuelven el socio o `null` [2]. Con un servicio real solo cambiaría esa capa por una petición POST.

El archivo de socios no contiene claves. Cada registro guarda una sal aleatoria y el resumen SHA-256 de la sal unida a la clave. Al ingresar, el navegador calcula el mismo resumen con la API Web Crypto (`crypto.subtle.digest`) y lo compara con el guardado [8]. Ante un e-mail inexistente o una clave incorrecta, la respuesta es la misma, E-mail o clave incorrectos, para no revelar qué cuentas existen [10]. Se trata de una simulación: el JSON es público, y un sistema real haría la comparación en el servidor, bajo HTTPS y con un algoritmo lento como `bcrypt` o `Argon2` [11].

La sesión se guarda en `sessionStorage`, que se borra al cerrar la pestaña [9]. Solo contiene el identificador, el nombre, el e-mail y la membresía; nunca la clave, la sal ni el resumen. El contenido que proviene de los JSON o de los formularios se inserta con `textContent`.

2.4 Modelo de audiencia

El primer paso de VSDM es clasificar a los usuarios según la información que necesitan. De los actores definidos en la AE1, esta entrega considera al visitante y al socio. Administrador y Entrenador quedan fuera del alcance porque requieren RF02 Roles y permisos y un servidor real.

Clase de usuario	Necesidades de información	Acceso
Visitante	Comparar planes, conocer precios y crear una cuenta.	Público, sin credenciales.
Socio	Lo mismo que el visitante, más el estado y el vencimiento de su membresía y una renovación rápida.	E-mail y clave verificados.
Administrador y Entrenador	Gestión de socios, pagos y rutinas.	Fuera del alcance de la AE2.



Socio es una especialización de Visitante: conserva todas sus vistas y agrega las propias. Por eso el catálogo de la tarjeta 3 sigue disponible para ambos y el socio no pierde el acceso público al autenticarse.

2.5 Fundamentación de VSDM

La consigna denomina VSDM al diseño guiado por las vistas de cada clase de usuario. En la literatura este enfoque aparece en WSDM (Web Site Design Method), de De Troyer y Leune, que parte del modelado de los usuarios y organiza el sitio según sus clases, y en VHDM (View-Based Hypermedia Design Methodology), de Lee, Kim, Kim y Cho, que obtiene las vistas a partir del modelo de datos [6, 7]. Ambos coinciden en que una misma información puede presentarse de manera distinta según quién la consulta. WSDM define un modelo de objetos por perspectiva, expresado como vista del modelo general, y una pista de navegación para cada una [6].

Fase	Aplicación en Gym Control
Clasificación de usuarios	Visitante y Socio, con Socio como especialización de Visitante.
Descripción de cada clase	Necesidades de información y forma de acceso de la tabla anterior.
Modelado de objetos	Socio, Membresía y Plan; la vista del socio es una proyección de esos objetos.
Diseño navegacional	Una pista de navegación por clase de usuario, representada en la Figura 4.
Diseño de implementación	Estilos de la AE1, estados de membresía con color y adaptación a teléfonos.
Implementación	acceso.html, bloque de js/app.js, sessionStorage y pruebas en navegador.

La vista del socio combina tres entidades: del socio toma el nombre y el identificador; de la membresía, el plan y el vencimiento; y del plan, su nombre, obtenido de data/planes.json. El estado es un dato derivado que se calcula al mostrar la vista. La sal y el resumen de la clave pertenecen al modelo de datos pero no forman parte de ninguna vista. La



aplicación no implementa un motor formal de VSDM: el método se usa para decidir qué información y qué enlaces corresponden a cada clase de usuario.

2.6 Diagrama de vistas y visibilidad

La Figura 4 presenta las dos vistas. Acceso y Mi cuenta son estados de la misma página acceso.html, como la búsqueda y los resultados de la tarjeta 3. El paso de una vista a la otra depende de la verificación de las credenciales o del cierre de la sesión.

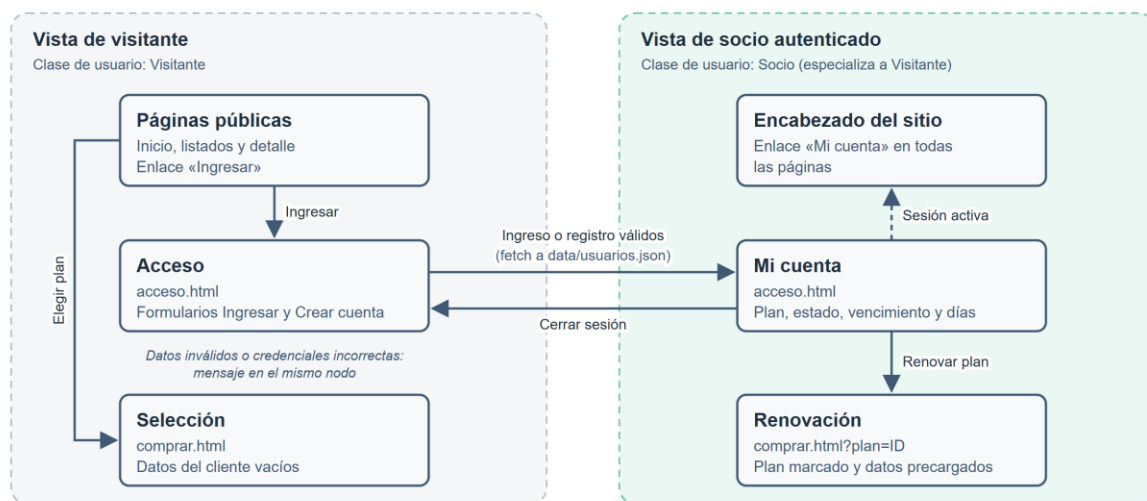


Figura 4 Vistas del visitante y del socio autenticado

La tabla siguiente resume qué elementos ve cada clase de usuario. El archivo editable docs/vistas-vsdm-tarjeta4.svg conserva el diagrama.

Elemento	Visitante	Socio
Catálogo, tabla y detalle de planes	Visible	Visible
Enlace del encabezado	Ingresar	Mi cuenta
Formularios Ingresar y Crear cuenta	Visible	Oculto
Plan, estado, vencimiento y días restantes	Oculto	Solo los propios
Formulario de compra	Datos vacíos	Plan marcado y datos precargados
Cerrar sesión	Oculto	Visible
Sal y resumen de la clave	Oculto	Oculto



2.7 Interacción, validaciones y estados

El evento submit de cada formulario llama a una función asíncrona que ejecuta preventDefault, valida los campos y solo entonces consulta al servidor simulado [4]. El evento input del formulario quita la marca de error del campo que se corrige, y el botón Cerrar sesión usa el evento click. Durante la verificación, el botón se deshabilita y muestra Verificando..., y el formulario indica aria-busy.

Situación	Respuesta en pantalla
Campo vacío o con solo espacios	Ingresa tu e-mail, tu clave o tu nombre, según el campo.
E-mail sin formato válido	Revisa el formato del e-mail.
Clave de menos de 8 caracteres	La clave debe tener al menos 8 caracteres.
Repetición distinta de la clave	Las claves no coinciden.
E-mail inexistente o clave incorrecta	E-mail o clave incorrectos.
E-mail ya registrado	Ya existe una cuenta con ese e-mail.
Error HTTP, de red o de formato	No se pudo verificar la cuenta; el usuario puede reintentar.

Los campos se validan con .trim() para detectar valores vacíos, pero la clave no se recorta porque los espacios forman parte de ella. Ante un error se informa solo el primero, el campo queda marcado con aria-invalid y recibe el foco. Los mensajes usan role="status" y aria-live para que los lectores de pantalla los anuncien. Las validaciones que no necesitan datos del servidor se resuelven antes de la petición, por lo que no generan tráfico.

2.8 Verificación

Se ejecutaron 35 comprobaciones en Google Chrome mediante un servidor HTTP local y una fecha fija, el 29 de septiembre de 2026, para que los estados de las membresías no dependan del día de la prueba. El procedimiento está en tests/acceso.cjs y el registro en docs/pruebas-tarjeta4.json. El comando npm test ejecuta también las 36 pruebas de la tarjeta 3, que siguen siendo correctas.



Caso representativo	Resultado observado
Ingreso con campos vacíos	Mensaje, campo marcado y foco; no se realiza la petición.
Claves distintas en el registro	Se informa que las claves no coinciden.
Clave incorrecta	Se consulta el JSON y se muestra el mensaje genérico.
Credenciales válidas	Vista del socio con Plan Semestral, Activa y 183 días restantes.
Renovar plan	comprar.html marca el plan, precarga los datos y muestra \$120.000.
Membresías vencida y por vencer	Se informan «Venció hace 29 días» y «4 días».
Error HTTP 404 o de red	Mensaje de error; la vista del socio no se habilita.
Vista de 390 píxeles	Sin desbordamiento horizontal; Elegir plan permanece visible.

El resultado global fue de 35 verificaciones correctas. La sesión no contiene la clave ni su resumen, la clave no aparece en la URL y un nombre con etiquetas HTML se muestra como texto. Las capturas se incluyen en los Anexos C a F.



3 TARJETA 11: FORMULARIO DE RECLAMOS Y CONSULTAS

Responsable: Martins, Santino

La tarjeta 11 corresponde a la opción Formulario de reseñas, opiniones o reclamos. Del conjunto de casos posibles que admite la tarjeta, esta parte se acota deliberadamente a los reclamos y consultas: una reseña o calificación de estrellas no resulta representativa del dominio de un gimnasio de gestión operativa como Gym Control, mientras que un canal de reclamos sí responde a un problema real (un equipo roto, un horario mal informado, una duda sobre el plan). Exige registrar los eventos con `addEventListener`, medir caracteres con `.length`, sanitizar la entrada con `.trim()` y enviar el comentario mediante `fetch` de forma asíncrona, mostrando la confirmación en el DOM. La fundamentación de la Unidad 1 corresponde a RNA (Red Neuronal Artificial), aplicada como proceso conceptual de procesamiento de lenguaje natural (PLN) y vectorización de texto.

El módulo incorpora a la ficha del Plan Mensual (`producto.html`) un formulario de reclamos y consultas. Conserva el diseño, los planes y los estilos existentes, y agrega un canal para que el socio reporte un problema puntual sin salir de la ficha del plan. No hay persistencia ni un backend real que reciba los reclamos.

3.1 Alcance y trazabilidad

Requisito	Evidencia en la implementación
Continuidad con AE1	El Plan Mensual, su ficha, la navegación y los estilos de Gym Control permanecen sin cambios.
Eventos desacoplados	<code>input</code> y <code>submit</code> se registran en <code>js/app.js</code> con <code>addEventListener</code> , sin <code>onclick</code> ni <code>onsubmit</code> .
HTTP y asincronía	<code>fetch</code> y <code>async/await</code> envían el reclamo a <code>data/confirmacion-reclamo.json</code> , que simula la respuesta del servidor.
Validación de texto	<code>.length</code> mide caracteres en vivo; <code>.trim()</code> descarta espacios; se exige un mínimo de 10 y un máximo de 500.
Metodología RNA	Pipeline conceptual de PLN para clasificar reclamos por categoría y prioridad.



3.2 Descripción del módulo

El usuario accede a `producto.html` desde el catálogo en cards o desde su enlace directo en el listado. Debajo de la ficha del Plan Mensual, una nueva sección presenta el formulario de reclamos: un campo de texto largo (textarea) y un botón de envío. A medida que el usuario escribe, el bloque de la tarjeta 11 en `js/app.js` cuenta los caracteres ingresados y actualiza un contador visible (0 / 500 caracteres), resaltándolo en rojo si se supera el límite.

Al enviar el formulario, el texto se normaliza con `.trim()` para descartar espacios en los extremos. Si el resultado tiene menos de 10 caracteres o más de 500, se informa el problema puntual y no se realiza ninguna petición HTTP. Si el texto es válido, el botón se deshabilita, se muestra “Enviando tu reclamo...” y se ejecuta una llamada `fetch` asíncrona contra `data/confirmacion-reclamo.json`, un archivo local que simula la respuesta que en producción devolvería un servidor. La respuesta se vuelca en un mensaje de confirmación y el formulario se vacía, listo para un nuevo reclamo.

3.3 Organización técnica

Componente	Responsabilidad
<code>producto.html</code>	Presenta la ficha del Plan Mensual y, debajo, la sección de reclamos y consultas.
<code>data/confirmacion-reclamo.json</code>	Simula la respuesta del servidor ante un envío exitoso.
<code>js/app.js</code>	Bloque de la tarjeta 11: cuenta caracteres, valida el texto, envía el reclamo y muestra la confirmación o el error.
<code>css/styles.css</code>	Estiliza el formulario (textarea, contador, mensajes) y lo adapta a la vista móvil.

El código de la tarjeta 11 es una función autoejecutable propia dentro de `js/app.js`, con el mismo criterio de aislamiento por tarjeta que usan las tarjetas 3 y 4: no modifica sus bloques ni los archivos `js/subtotal.js` y `js/cupon.js`. La comprobación de que no existan



atributos onclick ni onsubmit en ninguna página HTML del proyecto se incorporó también a esta batería de pruebas.

3.4 Escenario E02 de reclamos y consultas

Actor principal: socio o visitante que detecta un problema o tiene una consulta sobre el Plan Mensual. Precondición: el sitio se sirve mediante HTTP y el usuario se encuentra en la ficha del plan. Objetivo: registrar un reclamo o consulta y recibir una confirmación de que fue recibido.

Paso	Acción y respuesta esperada
1	Abrir producto.html; el sistema muestra la ficha del Plan Mensual y, debajo, el formulario de reclamos.
2	Escribir el reclamo; el contador de caracteres se actualiza en cada tecla mediante el evento input.
3	Enviar el formulario con menos de 10 caracteres; se informa el mínimo y no se realiza ninguna petición HTTP.
4	Completar el reclamo con un texto válido y enviarlo; el botón se deshabilita y se ejecuta la petición asíncrona.
5	Consultar la respuesta; se muestra el mensaje de confirmación y el formulario queda vacío.

Flujos alternativos: si el texto supera los 500 caracteres, se informa el máximo permitido y no se envía. Un reclamo compuesto solo por espacios se descarta mediante trim(). Si falla la petición HTTP (por ejemplo, un error 500 o un corte de red), se muestra un mensaje de error y el formulario permanece habilitado para reintentar el envío sin perder el texto escrito. La postcondición del flujo principal es la confirmación visual del reclamo; no se persiste en un servidor real ni se notifica a un encargado.

3.5 Fundamentación de RNA

Una Red Neuronal Artificial (RNA) es un modelo computacional compuesto por unidades simples (neuronas artificiales) organizadas en capas e interconectadas por pesos ajustables. McCulloch y Pitts formalizaron por primera vez la neurona artificial como una



unidad que combina entradas ponderadas y aplica una función de activación [12]; sobre esa base, las arquitecturas modernas se entrenan ajustando esos pesos para aproximar una función deseada.

Para que un texto libre como el de un reclamo pueda alimentar una RNA, primero debe convertirse en una representación numérica mediante un proceso de vectorización propio del Procesamiento de Lenguaje Natural (PLN): cada palabra o fragmento de texto se transforma en un vector de números que captura relaciones semánticas entre términos, tal como propusieron Mikolov et al. con su técnica de embeddings de palabras [13]. Ese vector es el que efectivamente recorre las capas de la red.

Aplicado a Gym Control, este proceso permitiría, en una etapa posterior fuera del alcance de la AE2, clasificar automáticamente cada reclamo por categoría (equipamiento, atención, horarios, cobros) y por prioridad, de modo que el encargado correspondiente lo reciba más rápido. La siguiente tabla traduce las fases habituales de un pipeline de PLN con RNA al caso concreto del formulario implementado.

Fase del pipeline RNA/PLN	Aplicación conceptual en Gym Control
Recolección de datos	El textarea de producto.html captura el texto libre del reclamo.
Preprocesamiento	trim() y una futura normalización (minúsculas, sin tildes) preparan el texto para tokenizar.
Vectorización	Cada token se representaría como un vector numérico (embedding) que captura su significado.
Arquitectura de la RNA	Capa de entrada (vectores del reclamo), capas ocultas y una capa de salida con una neurona por categoría.
Inferencia	La red asignaría una categoría (equipamiento, atención, horarios, cobros) y un nivel de prioridad.
Aplicación en el sistema	El reclamo ya clasificado se enrutaría al encargado correspondiente de Gym Control.

La implementación actual de la AE2 no ejecuta ninguna de estas fases: el texto se valida con reglas simples (longitud mínima y máxima) y se envía tal cual a un archivo JSON estático. La RNA se documenta aquí como el paso conceptual siguiente, coherente con el



alcance académico declarado en la introducción y con el mismo criterio que usó la tarjeta 3 al aclarar que SOHDM no se implementa como un motor formal (sección 1.5).

3.6 Mapa navegacional

El mapa deriva del escenario E02. La navegación existente Inicio → Catálogo dinámico → Detalle mensual se mantiene sin cambios. El formulario de reclamos es un estado de la propia página producto.html, no una página independiente: el contador, la validación y el envío ocurren sin recargar ni navegar. El archivo editable docs/mapa-navegacional-tarjeta11.mmd conserva el diagrama.

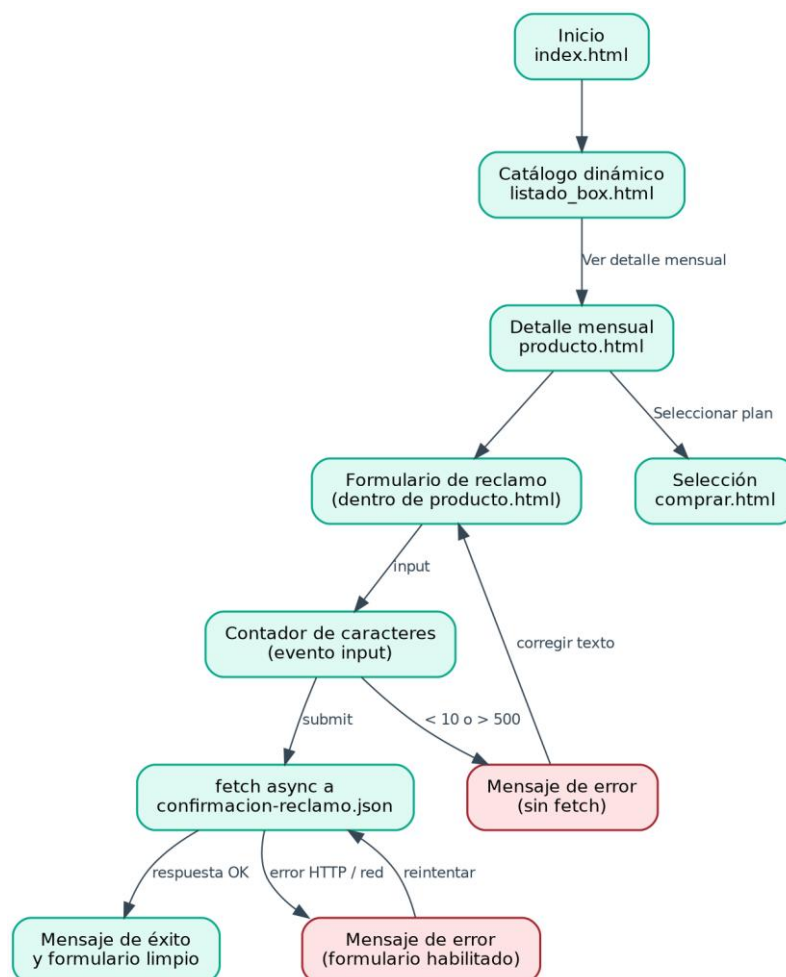


Figura 9 Mapa navegacional del escenario de reclamos y consultas



3.7 Interacción y estados de la interfaz

El evento input del campo de texto llama a `actualizarContador()`, que recalcula la longitud en cada tecla y resalta el contador cuando se supera el límite de 500 caracteres. El evento submit del formulario llama a `enviarReclamo()`, que aplica `trim()`, valida el mínimo y el máximo, y solo si el texto es válido ejecuta la petición HTTP. Todos se vinculan mediante `addEventListener` en `js/app.js`; el HTML no contiene atributos `onclick` ni `onsubmit`.

El mensaje de estado usa `role="status"` y `aria-live="polite"` para comunicar tanto los errores de validación como la confirmación del envío sin necesidad de recargar la página. Mientras se espera la respuesta, el botón de envío se deshabilita para evitar reclamos duplicados. El CSS reordena el formulario en una columna para pantallas de teléfono, reutilizando los mismos puntos de quiebre que el resto del sitio.

3.8 Verificación

Se ejecutaron 12 comprobaciones en Google Chrome mediante un servidor HTTP local, con el mismo procedimiento de prueba automatizada (Playwright) que usan las tarjetas 3 y 4. El registro completo se conserva en `docs/pruebas-tarjeta11.json` y el procedimiento reproducible en `tests/reclamos.cjs`. Las pruebas cubren el contador de caracteres, la validación de mínimo y máximo, el envío exitoso, el manejo de errores HTTP, el reintento y la vista móvil.



Caso verificado	Resultado
El contador inicia en 0 / 500 caracteres	Correcto
El contador reacciona al evento input sin enviar el formulario	Correcto
Menos de 10 caracteres bloquea el envío sin llamar a fetch	Correcto
Un reclamo de solo espacios se rechaza por trim()	Correcto
Un reclamo válido dispara un único fetch asíncrono	Correcto
El formulario se limpia tras la confirmación	Correcto
Más de 500 caracteres se rechaza y resalta el contador	Correcto
Sin onclick ni onsubmit en las páginas HTML	Correcto
Error HTTP 500 muestra el mensaje de error, no bloquea el formulario	Correcto
Reintentar el envío después del error funciona	Correcto
Diseño móvil de 390 px sin desbordamiento horizontal	Correcto
Recorrido normal sin errores JavaScript	Correcto

El resultado global fue de 12 verificaciones correctas. Las capturas se incluyen en los Anexos G a I.

CONCLUSIONES FINALES

La Actividad Evaluativa 2 incorpora a Gym Control tres módulos con JavaScript sobre el proyecto de la AE1. Los tres cumplen el requisito transversal: registran los eventos con `addEventListener` en `js/app.js`, sin atributos `onclick` ni `onsubmit`, y obtienen sus datos con `fetch` y `async/await` desde archivos JSON locales.

La tarjeta 3 agrega un buscador distinto del subtotal y de la validación del cupón desarrollados previamente, con filtrado de `Array.filter` en memoria y actualización inmediata del DOM. El escenario E01 permitió justificar los campos de consulta, los estados de resultados y los accesos entre catálogo y formulario, y el mapa navegacional SOHDM relaciona esa interacción con las pantallas existentes.

La tarjeta 4 agrega la distinción entre visitante y socio, con validación de claves y una verificación asíncrona de credenciales. VSDM permitió decidir, antes de programar, qué información corresponde a cada clase de usuario: el visitante conserva el catálogo público y el socio suma su membresía y una renovación con sus datos cargados.



La tarjeta 11 agrega un canal de reclamos y consultas en la ficha del Plan Mensual, con validación de texto en el cliente, un envío asíncrono simulado y confirmación inmediata en el DOM. El escenario E02 permitió justificar el campo de texto, sus estados de error y la confirmación de envío, y la fundamentación de RNA relaciona ese texto libre con un futuro clasificador de reclamos por categoría y prioridad, sin que la implementación actual ejecute ese procesamiento.

Las pruebas en navegador confirman los casos esperados y las rutas de error: 36 comprobaciones de la tarjeta 3, 35 de la tarjeta 4 y 12 de la tarjeta 11. Los datos son de ejemplo y tanto la verificación de credenciales como el envío de reclamos son simulaciones; para operar membresías, cuentas y reclamos reales serían necesarios un servidor, persistencia, HTTPS, almacenamiento seguro de claves, notificación al personal y la gestión de roles prevista en RF02.



BIBLIOGRAFÍA

- [1] Lee, H., Lee, C., & Yoo, C. (1999). A scenario-based object-oriented hypermedia design methodology. *Information & Management*, 36(3), 121-138.
[https://doi.org/10.1016/S0378-7206\(99\)00011-7](https://doi.org/10.1016/S0378-7206(99)00011-7)
- [2] MDN Web Docs. (s. f.). Using the Fetch API. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- [3] MDN Web Docs. (s. f.). Array.prototype.filter(). https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/filter
- [4] MDN Web Docs. (s. f.). EventTarget.addEventListener().
<https://developer.mozilla.org/es/docs/Web/API/EventTarget/addEventListener>
- [5] Tarnoski, S., Martins, S., & Olexyn, F. (2026). Gym Control: análisis y diseño de la Actividad Evaluativa 1. Repositorio del proyecto.
<https://github.com/santiagotarnoski/gym-control-ae1>
- [6] De Troyer, O. M. F., & Leune, C. J. (1998). WSDM: A user centered design method for Web sites. *Computer Networks and ISDN Systems*, 30(1-7), 85-94.
[https://doi.org/10.1016/S0169-7552\(98\)00042-7](https://doi.org/10.1016/S0169-7552(98)00042-7)
- [7] Lee, H., Kim, J., Kim, Y. G., & Cho, S. H. (1999). A view-based hypermedia design methodology. *Journal of Database Management*, 10(2), 3-13.
<https://doi.org/10.4018/jdm.1999040101>
- [8] MDN Web Docs. (s. f.). SubtleCrypto: digest() method. <https://developer.mozilla.org/en-US/docs/Web/API/SubtleCrypto/digest>
- [9] MDN Web Docs. (s. f.). Window: sessionStorage property.
<https://developer.mozilla.org/en-US/docs/Web/API/Window/sessionStorage>
- [10] OWASP Foundation. (s. f.). Authentication cheat sheet. OWASP Cheat Sheet Series.
https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html



-
- [11] OWASP Foundation. (s. f.). Password storage cheat sheet. OWASP Cheat Sheet Series.
https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- [12] McCulloch, W. S., & Pitts, W. (1943). A logical calculus of the ideas immanent in
nervous activity. *Bulletin of Mathematical Biophysics*, 5(4), 115–133.
<https://doi.org/10.1007/BF02478259>
- [13] Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word
representations in vector space. arXiv:1301.3781. <https://arxiv.org/abs/1301.3781>



ANEXOS

Anexo A Evidencia de búsqueda

Buscá el plan que se adapte a vos

Buscar por nombre, descripción o categoría

compromiso



Duración del plan

12 meses

1 de 4 planes encontrados.

Compromiso

Plan Anual
12 meses

\$ 216.000
/ total

La opción de mayor duración y
mejor valor mensual.

Elegir plan

Figura 2 Búsqueda por categoría y duración con un resultado



Anexo B Estado sin resultados

Buscá el plan que se adapte a vos

Buscar por nombre, descripción o categoría

compromiso



Duración del plan

1 mes

0 de 4 planes encontrados.

No encontramos planes con esos filtros. Probá otra búsqueda o limpiá los filtros.

Figura 3 Filtros incompatibles y mensaje al usuario

Anexo C Validación de claves en el registro

02 Crear cuenta

Nombre y apellido

Ana Gómez

E-mail

ana@correo.com

Clave

.....

Mínimo 8 caracteres.

Repetir clave

.....

Crear cuenta

Las claves no coinciden.

Figura 5 Claves que no coinciden en el formulario Crear cuenta



Anexo D Credenciales incorrectas

01 Ingresar

E-mail

lucia.benitez@gymcontrol.test

Clave

Ingresar

E-mail o clave incorrectos.

Figura 6 Respuesta genérica tras la verificación con fetch

Anexo E Vista del socio autenticado

SOCIO S-0012

Hola, Lucía

PLAN Plan Semestral	ESTADO Activa
VENCIMIENTO 31 de marzo de 2027	DÍAS RESTANTES 183 días

Tu membresía está al día.

Renovar plan Cerrar sesión

Figura 7 Membresía activa del socio de prueba



Carrera: Ingeniería en Sistemas de
Información
Materia: Paradigmas y Lenguajes de
Programación III
Estudiantes: Martins, Olexyn y Tarnoski

Profesor: Sicardi, Dante
Comisión: A
Actividad Evaluativa 2

Anexo F Renovación con datos precargados

01 Datos del cliente

Nombre del cliente

Lucía Benítez

Dirección

Calle, número y localidad

Teléfono

Ej.: 3764 123456

E-mail

lucia.benitez@gymcontrol.test

Figura 8 Formulario de compra con los datos del socio

Anexo G Envío exitoso de un reclamo

Tu reclamo o consulta

Contanos qué pasó, por ejemplo un problema con un equipo o con el horario de atención...

0 / 500 caracteres

Enviar reclamo

¡Gracias! Recibimos tu reclamo y el equipo de Gym Control lo va a revisar a la brevedad.

Figura 10 Confirmación del reclamo tras un envío válido

